

UEFI - Hello Friend - 1

Tuesday, August 25, 2026 7:31 PM

Source : <https://www.rodsbooks.com/efi-programming/hello.html>
https://uefi.org/specs/UEFI/2.10/04_EFI_System_Table.html

Un simple hello world :

On initialise les deux paramètres pour le efi_main qui sont l'image handle donc notre identifiant et le pointeur vers le system handle. On utilise la convention System V (SysV) qui est utilisé par Linux (GCC).

Le problème est que UEFI utilise la convention d'appelle de microsoft donc il faut utiliser la même convention lorsque le programme se compile.

Pour cela on va utiliser cette variable **EFIAPI**. Ce mot clé dit au compilateur que pour cette fonction précisément on utilise la convention microsoft et pas celle de linux.

Si on utilisait EDK 2 on aurait pas besoin de justifier ça. Mais ici on code avec la bibliothèque GNU-EFI.

Donc **EFIAPI** force le compilateur à utiliser la convention d'appel **Microsoft** (celle de l'UEFI) au lieu de celle de **Linux** (SysV), pour les fonctions appelées par l'UEFI (comme efi_main)

```
#include <efi.h>
#include <efilib.h>
```

```
//ici on dit au compilateur de compiler ce programme avec la convention
d'appelle "microsoft" et pas linux.
```

```
/* cette initialisation est similaire à celle-ci :
```

```
Ici on définit un type NTSTATUS tous en imposant la convention d'appelle
NTAPI de windows (NTAPI : La convention d'appel (équivalent de __stdcall), qui
indique comment les paramètres sont passés sur la pile.)
```

```
typedef NTSTATUS (NTAPI* fnNtOpenProcess)(
    _Out_ PHANDLE ProcessHandle,
    _In_ ACCESS_MASK DesiredAccess,
    _In_ PCOBJECT_ATTRIBUTES ObjectAttributes,
    _In_opt_ PCLIENT_ID ClientId
);
*/
```

```
//on définit le type de retour de efi_main qui est un efi_status et on lui impose
la convention EFIAPi (microsoft)
```

```
EFI_STATUS
EFIAPI
efi_main (EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE * SystemTable) {
```

```
//ici le EFI_HANDLE est un pointeur sur le context du firmware donc comme on
a dit sur l'ID du programme.
```

```
// le EFI_SYSTEM_TABLE est un pointeur sur le system table de l'UEFI qui va
pointer sur d'autre table.
```

Ici on va jouer avec les interfaces qui contiennent des pointeurs de fonction qui

vont nous permettre de faire certain trucs.

Ici on va simplement afficher un texte.

//d'abord on doit récupérer un pointeur sur la table **simple_text_output**. Ici on va "partager" nos informations (image et le systemTable) au fonction interne de GNU-EFI pour nous faciliter la tâche et éviter d'utiliser des fonctions internes à l'UEFI. Donc ici on pourrait juste utiliser Print() et ça fonctionnerait parfaitement bien. Mais comme on veut utiliser des fonctions natives de l'UEFI on n'a pas forcément besoin d'utiliser **InitializeLib**.

Il faut juste savoir que cette fonction **InitializeLib(image, systab)** initialise les rouages internes de GNU-EFI en lui donnant accès à notre image et à la System Table. Elle est optionnelle si on n'appelle que des fonctions UEFI natives (comme dans cet exemple avec OutputString), mais obligatoire dès qu'on utilise les fonctions d'aide de GNU-EFI (comme Print).

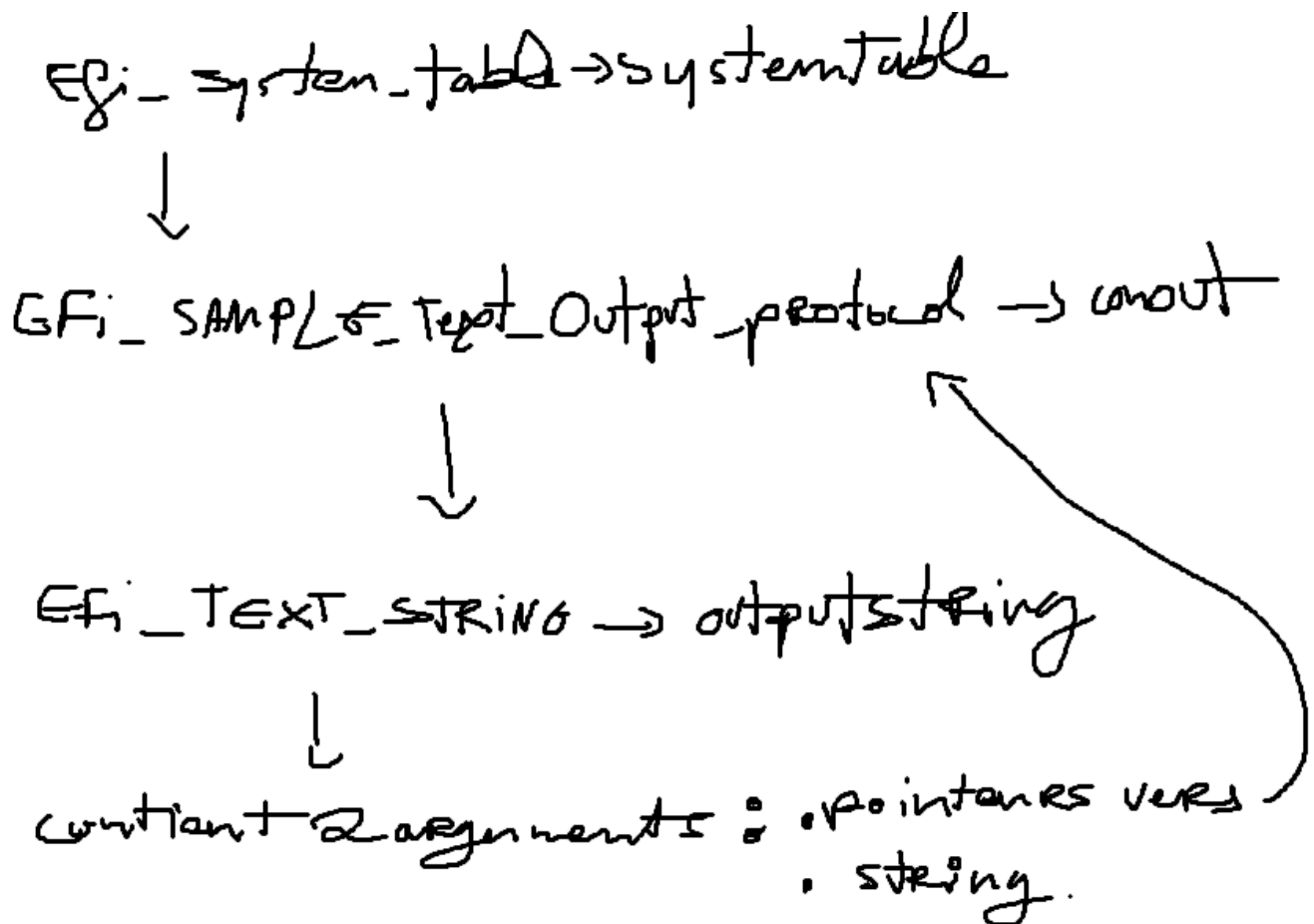
Par bonne pratique, on la met toujours en début de efi_main pour éviter les crashes difficiles à diagnostiquer.

InitializeLib(image, SystemTable)

Le prototype de la structure **EFI_SYSTEM_TABLE** ressemble à ceci :

```
#define EFI_SYSTEM_TABLE_SIGNATURE 0x5453595320494249
#define EFI_2_100_SYSTEM_TABLE_REVISION ((2<<16) | (100))
#define EFI_2_90_SYSTEM_TABLE_REVISION ((2<<16) | (90))
#define EFI_2_80_SYSTEM_TABLE_REVISION ((2<<16) | (80))
#define EFI_2_70_SYSTEM_TABLE_REVISION ((2<<16) | (70))
#define EFI_2_60_SYSTEM_TABLE_REVISION ((2<<16) | (60))
#define EFI_2_50_SYSTEM_TABLE_REVISION ((2<<16) | (50))
#define EFI_2_40_SYSTEM_TABLE_REVISION ((2<<16) | (40))
#define EFI_2_31_SYSTEM_TABLE_REVISION ((2<<16) | (31))
#define EFI_2_30_SYSTEM_TABLE_REVISION ((2<<16) | (30))
#define EFI_2_20_SYSTEM_TABLE_REVISION ((2<<16) | (20))
#define EFI_2_10_SYSTEM_TABLE_REVISION ((2<<16) | (10))
#define EFI_2_00_SYSTEM_TABLE_REVISION ((2<<16) | (00))
#define EFI_1_10_SYSTEM_TABLE_REVISION ((1<<16) | (10))
#define EFI_1_02_SYSTEM_TABLE_REVISION ((1<<16) | (02))
#define EFI_SPECIFICATION_VERSION
EFI_SYSTEM_TABLE_REVISION
#define EFI_SYSTEM_TABLE_REVISION EFI_2_100
_SYSTEM_TABLE_REVISION
typedef struct {
    EFI_TABLE_HEADER          Hdr;
    CHAR16                    *FirmwareVendor;
    UINT32                     FirmwareRevision;
    EFI_HANDLE                 ConsoleInHandle;
    EFI_SIMPLE_TEXT_INPUT_PROTOCOL *ConIn;
    EFI_HANDLE                 ConsoleOutHandle;
    EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL *ConOut;
    EFI_HANDLE                 StandardErrorHandle;
    EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL *StdErr;
    EFI_RUNTIME_SERVICES       *RuntimeServices;
    EFI_BOOT_SERVICES          *BootServices;
    UINTN                      NumberOfTableEntries;
    EFI_CONFIGURATION_TABLE     *ConfigurationTable;
} EFI_SYSTEM_TABLE;
```

À partir de l'adresse <<https://uefi.org/specs/UEFI/2.10/04>



Structure de EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL :

```
typedef struct _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL {  
    EFI_TEXT_RESET Reset;  
    EFI_TEXT_STRING OutputString;  
    EFI_TEXT_TEST_STRING TestString;  
    EFI_TEXT_QUERY_MODE QueryMode;  
    EFI_TEXT_SET_MODE SetMode;  
    EFI_TEXT_SET_ATTRIBUTE SetAttribute;  
    EFI_TEXT_CLEAR_SCREEN ClearScreen;  
    EFI_TEXT_SET_CURSOR_POSITION SetCursorPosition;  
    EFI_TEXT_ENABLE_CURSOR EnableCursor;  
    SIMPLE_TEXT_OUTPUT_MODE *Mode;  
} EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL;
```

Ce schéma correspond à ceci :

```
SystemTable->ConOut->OutputString(SystemTable->ConOut, L"Hello UEFI\r\n");
```

On pourrait écrire de cette manière mais ça ne répondrait pas à la convention d'appel microsoft ABI. Il faudrait alors le compiler avec une option (ce qui est possible sur les compilateur récent). En fait tout dépend de comment on compile notre programme. J'ai vu que les compilateur moderne peuvent directement traduire tout en format ABI.

Pour cet exemple on va quand même utiliser cette fonction qui vient de la librairie efi.h : `uefi_call_wrapper()`

Cette fonction prend en parametre le nom de la fonction qui va être utiliser, le nombre de paramètre que cette fonction va utiliser et les paramètres. Ce qui nous donne :

Comme la fonction **OutputString** prend les deux paramètres suivants :

```
typedef
EFI_STATUS
(EFIAPI *EFI_TEXT_STRING) (
    IN EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL *This,
    IN CHAR16 *String
);
```

```
uefi_call_wrapper(conout->OutputString, 2, conout, L"Hello Friend !\n");
```

Et enfin on retourne un code success avec EFI_STATUS ce qui nous donne :

```
return EFI_SUCCESS;

}
```

En bref notre premier programme ressemble à ceci :

```
#include <efi.h>
#include <efilib.h>

EFI_STATUS EFIAPI efi_main (EFI_HANDLE ImageID, EFI_SYSTEM_TABLE *
systemTable){

SIMPLE_TEXT_OUTPUT_INTERFACE * conout; // on définit un pointeur sur
la structure SIMPLE_TEXT_OUTPUT__INTERFACE.

conout = systemTable->ConOut;
//Initializelib(imageID, SystemTable);
uefi_call_wrapper(conout->OutputString, 2, conout, L"Hello Friend !\n");

return EFI_SUCCESS;

}
```